

0.15. Deux rythmes, un seul chronomètre

Le même programme en plus court : un seul compteur, et le modulo fait le reste.

1. Ce qu'il faut comprendre

C'est le programme du 0.14, en plus simple. L'écran est exactement le même : la lettre du bas change quatre fois pendant que celle du haut change une fois.

Un seul chronomètre : au lieu de deux compteurs, `images` compte de 0 à 59, puis repart à 0. Il fait donc un tour complet **chaque seconde**.

La lente change quand le chronomètre repasse par **0** : une fois par tour, donc une fois par seconde.

La rapide change quand `images % 15 == 0` : quand `images` est un **multiple de 15**, c'est-à-dire 0, 15, 30 ou 45. Le reste de la division par 15 ne vaut 0 que pour ces nombres : $30 \% 15 = 0$, mais $31 \% 15 = 1$. Cela fait **4 fois** par tour, donc 4 fois par seconde.

Le retour à A sans `if` : `(rapide + 1) % sizeof(ALPHABET)` ajoute 1, puis garde le reste de la division par 26. Tant qu'on est sous 26, le reste est le nombre lui-même ($7 \% 26 = 7$) ; à 26, il retombe à **0** ($26 \% 26 = 0$) : après Z, A.

Ce qu'on gagne : 3 variables au lieu de 4, deux `if` d'une ligne au lieu de deux blocs, et un programme plus léger. **Ce qu'on perd** : les deux rythmes sont liés au même chronomètre ; au 0.14, on pouvait choisir n'importe quelle durée pour chacun, sans rapport entre eux.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1438 octets** de cartouche.

1. UNE VARIABLE — LENTE

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
```

2. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
```

3. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

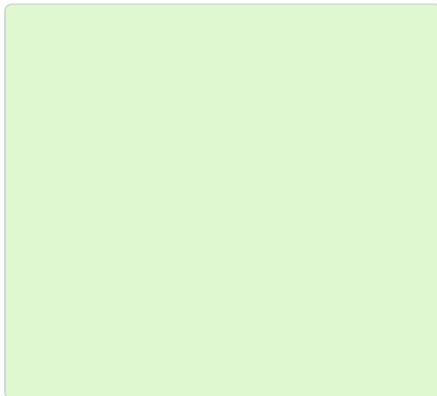
        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        // images % 15 = le reste de images divisé par 15 ;
        // il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        // et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

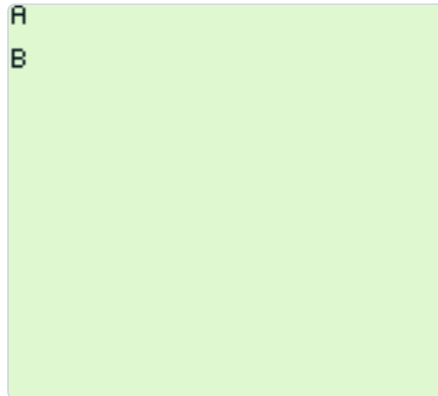
        poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide
    }
}
```

3. Ce qui se passe, étape par étape

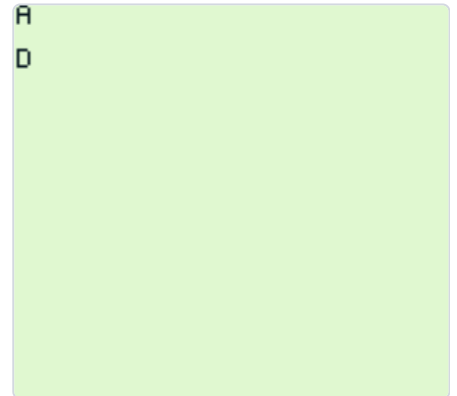
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



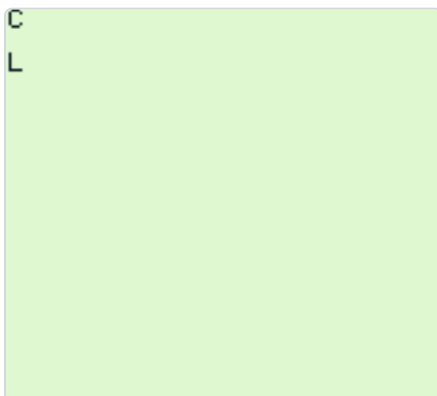
la toute première image — la console vient de démarrer — image 1



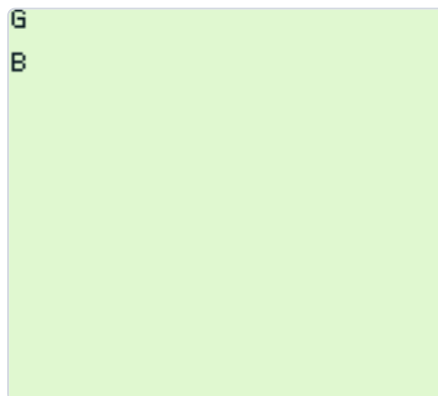
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Comme au 0.14 : la lettre du bas change quatre fois pendant que celle du haut change une fois.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la rapide change à chaque quart de seconde (8 fois en 2 s) (AB AC AD BE BF BG BH CI CJ)
- ✓ la lente change toutes les secondes (2 fois en 2 s)
- ✓ après Z, on revient bien à A (les lettres restent des lettres)

Ensuite : **0.15.1. Deux rythmes, un seul chronomètre — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++